Synergistic Integration of Machine Learning and Mathematical Optimization for Unit Commitment

Jianghua Wu, *Student Member, IEEE*, Peter B. Luh, *Life Fellow, IEEE*, Yonghong Chen, *Fellow, IEEE*, Bing Yan, *Member, IEEE*, and Mikhail A. Bragin, *Senior Member, IEEE*

***Abstract*—Unit Commitment (UC) is important for power system operations. With increasing challenges, e.g., growing intermittent renewables and intra-hour net load variability, traditional mathematical optimization could be time-consuming. Machine learning (ML) is a promising alternative. However, directly learning good solutions is difficult in view of the combinatorial nature of UC. This paper synergistically integrates ML within our recent decomposition and coordination method of Surrogate Lagrangian Relaxation to learn "good enough" subproblem solutions of deterministic UC. Compared to original UC, a subproblem is much easier to learn. Nevertheless, predicting good-enough subproblem solutions is still challenging because of the "jumps" of binary decisions and many types of constraints. To overcome these issues, subproblem dimensionality is reduced via aggregating multipliers. Multiplier distributions are novelly specified based on "jumps" for effective learning. Loss functions are innovatively designed to improve prediction qualities. Ordinal Optimization and branch-and-cut are used as backups for unfamiliar cases. Furthermore, online self-learning is seamlessly integrated with offline learning to exploit solutions from daily operations. Results on the IEEE 118-bus system and the Polish 2383-bus system demonstrate that continual learning keeps on improving the subproblem-solving process with near-optimality of the overall solutions maintained. Our method opens a new direction to solve complicated UC.**

***Index Terms*—Deep neural network, machine learning, mixed-integer linear programming, surrogate Lagrangian relaxation, unit commitment.**

Manuscript received 25 April 2022; revised 24 August 2022 and 12 December 2022; accepted 19 January 2023. Date of publication 26 January 2023; date of current version 26 December 2023. This work was supported in part by the Midcontinent ISO, in part by the National Science Foundation under Grants ECCS-1810108 and ECCS-1831811, and in part by the National Taiwan University under Yushan Scholar Program under Grants 110VV003 and 111VV003. Paper no. TPWRS-00589-2022. *(Corresponding author: Jianghua Wu.)*

Jianghua Wu is with the Department of Electrical and Computer Engineering, University of Connecticut, Storrs, CT 06269 USA (e-mail: jianghua.wu@uconn.edu).

Peter B. Luh, deceased, was with the Department of Electrical and Computer Engineering, University of Connecticut, Storrs, CT 06269 USA, and also with the Department of Electrical Engineering, National Taiwan University, Taipei 10617, Taiwan (e-mail: peter.luh@uconn.edu).

Yonghong Chen is with the MISO, Carmel, IN 46032 USA (e-mail: ychen@misoenergy.org).

Bing Yan is with the Department of Electrical and Microelectronic Engineering, Rochester Institute of Technology, Rochester, NY 14623 USA (e-mail: bxyeee@rit.edu).

Mikhail A. Bragin was with the Department of Electrical and Computer Engineering, University of Connecticut, Storrs, CT 06269 USA. He is now with the Department of Electrical and Computer Engineering, University of California, Riverside, CA 92507 USA (e-mail: mbragin@engr.ucr.edu).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TPWRS.2023.3240106.

Digital Object Identifier 10.1109/TPWRS.2023.3240106

I. Introduction

UNIT Commitment (UC) is an important problem in power system operations. It determines units' commitment statuses and generation levels to meet the system demand while minimizing the total operating cost subject to unit-level and system-wide constraints. UC has generally been formulated as a Mixed-Integer Linear Programming (MILP) problem, and is believed to be NP-hard. It is solved on a daily basis by Independent System Operators (ISO), and its cost can be up to $50M per day for a large ISO. Obtaining high-quality UC solutions within a strict time limit (e.g., 20 minutes) is thus crucial for the efficient operation of power systems. Although current MILP solvers based on branch-and-cut (B&C) [1] are powerful, UC is facing challenges. Increased penetration of intermittent renewables leads to increased uncertainties and growing intra-hour net load variability. To address these, sub-hourly UC has been suggested to improve system flexibility, the increased number of periods and reduced ramping capabilities per period make the problem larger and more complicated. The complexity could be further exacerbated if large numbers of virtual transactions, combined cycle units, or distributed energy resources are considered. Obtaining good solutions could be difficult when facing these new challenges. Machine learning (ML) is a promising alternative. Recently, multiple "indirect" ML methods for UC have been presented, e.g., learning effective branching strategies for B&C or identifying and removing inactive transmission constraints [2], [3], [4], [5], [6], [7], [8]. "Direct" methods have also been explored, e.g., using graph neural networks, reinforcement learning, and gated recurrent unit [9], [10], [11], [12], [13], [14] to directly solve UC problems. These methods introduced learning to UC. Nevertheless, in view of the combinatorial nature of UC with an exponentially growing number of possible solutions, indirect methods may not be effective and direct methods may not be able to provide high-quality solutions for large problems, especially for unfamiliar cases.

This paper presents a synergistic integration of ML and mathematical optimization by embedding Deep Neural Networks (DNNs) within our recent decomposition and coordination method of Surrogate Lagrangian Relaxation (SLR) [15] for deterministic hourly UC problems. Decomposition and coordination approaches are promising since a subproblem after decomposition enjoys an exponential reduction of complexity as compared to that of the original problem. Traditional Lagrangian Relaxation (LR), however, suffers from major difficulties of significant computational efforts per iteration and zigzagging of

multipliers [16]. Recently, SLR overcame these difficulties, and in particular, only requires subproblem solutions to be "good enough", i.e., feasible with respect to subproblem constraints and satisfying a simple convergence condition based on the "Surrogate Lagrangian". SLR has been further improved by embedding the Ordinal Optimization (OO) concepts to quickly obtain good-enough subproblem solutions by modifying solutions from previous iterations or by solving crude subproblems [17]. Nevertheless, there is no learning, and SLR+OO may not be sufficient when facing new challenges.

In Section II, two relevant optimization methods, B&C and LR, are reviewed. Existing ML methods for UC are also examined. In Section III, the UC formulation is summarized, with system demand and transmission capacity constraints considered but not reserve requirements for simplicity. Key steps of SLR [15] are then presented.

In Section IV, ML is used to learn SLR subproblem solutions. For simplicity, only system demand and unit initial statuses are assumed to change from day to day. The set of units, unit characteristics, and capacities of transmission lines are assumed unchanged across days. Under these simplifying assumptions, a fully connected multilayer perceptron is adopted rather than advanced DNN architectures, and its task is to predict good-enough subproblem solutions based on a given set of multipliers and unit initial statuses. Although the size of a subproblem is much smaller than that of the original problem, learning to predict good-enough subproblem solutions is still challenging. First, large numbers of multipliers and variables require a large ML model, which may be hard to train. Second, because binary decisions jump at a few breakpoints of multipliers, the distribution of random multipliers used in training must be carefully designed for effective training. Third, it is difficult for ML predictions to be feasible to many types of constraints. Finally, predictions may not be good enough when facing unfamiliar cases. To address the above difficulties, "aggregated" multipliers are used and redundant variables are removed to reduce the dimensionality of the multilayer perceptron. Distributions of random multipliers are also novelly specified based on the locations of "jumps" of binary decisions. Moreover, a loss function considering target values and constraint violations is designed to improve the quality of predictions. Finally, graceful degradation is accomplished for unfamiliar cases by using OO or B&C as a backup, with a slight increase in computational time. The DNNs are trained through offline supervised learning via back-propagation, where good-enough subproblem solutions obtained from B&C are used as learning targets. Also, in view of the shadow price concept of multipliers, DNN predictions are interpretable.

In Section V, supplementary online self-learning is developed to effectively utilize subproblem solutions available from daily operations. In daily operations, there are "positive cases" where ML successfully predicts good-enough subproblem solutions; and "negative cases" where ML fails and OO or B&C is called upon. Positive cases can be used to augment learning by using the loss function in offline learning with predicted solutions as targets. For negative cases, although ML predictions are not good enough, a loss function to improve the "good-enoughness" is innovatively designed, and its subgradient is derived and used in back-propagation to improve DNN weights. Good-enough solutions obtained from OO or B&C can also be further learned similar to that for positive cases. A unified learning process is thus established at the simple switching of the loss functions.

In Section VI, three examples are presented. In Example 1, results on a four-unit problem show that ML learns to predict good-enough subproblem solutions. For unfamiliar cases, OO or B&C maintains the quality of the overall solution as a backup. In Example 2, results on the IEEE 118 bus system demonstrate that ML accelerates the subproblem-solving process and that the near-optimality of the overall solutions is maintained. Also, with continual learning, ML provides an increasing percentage of good-enough subproblem solutions, leading to further reductions of subproblem-solving times. In Example 3, results on the Polish 2383-bus system demonstrate the ability of our approach to solve large problems. Although reserve constraints are not considered in this first attempt to integrate ML and SLR for simplicity, they can be incorporated into the current framework with a slight increase in training time. For this first attempt to integrate ML and SLR, our goal is not to outperform B&C in terms of solution quality or computation efficiency. For very complex UC problems, e.g., MISO's problem where B&C suffers from poor performance (e.g., [17] and [19]), however, our approach would present a promising new direction.

II. Literature Review

Section II-A reviews two mathematical optimization methods of B&C and Lagrangian relaxation to solve UC problems. Section II-B reviews existing indirect and direct ML methods to solve UC problems.

A. Mathematical Optimization Methods

1) Branch-and-Cut [1]: B&C is the state-of-the-art and practice MILP method used in most commercial optimization solvers, e.g., CPLEX and Gurobi. When solving a problem, it first applies "valid cuts" trying to delineate the convex hull (the smallest set containing all feasible solutions). Since the optimal solution to a linear problem is located at a vertex of the convex hull, if the convex hull or the facet containing the optimal solution is obtained, then the problem is essentially solved. When the convex hull is difficult to obtain, the method relies on time-consuming branch-and-bound and heuristics to search for near-optimal solutions. B&C has shown advantages in terms of solution quality and computation speed for many applications. For complicated UC, however, it may have difficulties obtaining near-optimal solutions as shown in [17] and [19].

2) Lagrangian Relaxation: Lagrangian Relaxation (LR) [16] was one of the early methods to solve UC problems. It relaxes system-wide constraints and decomposes the relaxed problem into subproblems for an exponential reduction of complexity. Subproblem solutions are then coordinated by iteratively updating multipliers based on subgradients. Standard LR, however, suffers from major difficulties, e.g., significant computational requirements (solving all subproblems to obtain a subgradient), and poor coordination (guesstimating the unknown optimal dual value to compute step sizes, and the zigzagging of multipliers). Recently, SLR [15] overcame the above-mentioned difficulties. Within SLR, only one subproblem needs to be solved for the

updating of multipliers, and that subproblem only needs to be solved "good enough", i.e., its solution is feasible in terms of subproblem constraints and satisfies a simple convergence condition – the "Surrogate Optimality Condition" (SOC, see (10) later). Moreover, since "surrogate" subgradients do not change drastically from one iteration to the next and guesstimating the optimal dual value is no longer needed to compute step sizes, SLR has smoother multiplier trajectories and much-improved coordination as compared to those of standard LR. In [19], good-enough subproblem solutions were obtained by using B&C (SLR+B&C). In [17], SLR has been further improved by adding absolute-value penalties on constraint violations (based on [20]) for accelerated convergence; and by utilizing the Ordinal Optimization (OO) concepts [21] to obtain good-enough subproblem solutions quickly through modifying solutions from previous iterations or through solving crude subproblems for a major reduction of subproblem-solving times (SLR+OO).

B. ML Methods for UC

This subsection reviews the methods that indirectly or directly use the learning concept to solve UC problems.

1) Indirect Methods: Recently, multiple methods [2], [3], [4], [5], [6], [7], [8] used ML to enhance the performance of B&C. In [2] and [3], ML constructed branching strategies through supervised learning to speed up the branch-and-bound process. For large problems, in view of a large number of possible branching choices, the method may not be effective. In [4], k-nearest neighbor (KNN) was used to identify and remove inactive transmission capacity constraints for different scenarios. In [5], the unconstrained problem was solved at first. Violated constraints were then added back, and the problem was resolved. The above process was repeated until no new constraint is violated. KNN was then used to learn these active constraints for different scenarios. This method is limited because it is difficult to design a small number of representative scenarios. Another classifier was developed in [7] to identify if a UC instance is "easy" or "hard." For an easy instance, an MILP solver is used to find the optimal solution. For a hard instance, heuristic techniques are used to speed up the MILP solver at the sacrifice of solution quality. Reference [8] reduced the number of binary variables by learning to fix the on/off statuses of certain units under different scenarios for the Polish system**.** Nevertheless, it would be difficult to consider enough scenarios, and prediction for instances beyond existing scenarios may not be promising.

2) Direct Methods: Advanced ML techniques have been explored to directly learn UC solutions [9], [10], [11], [12], [13], [14]. Graph Convolutional Network [9] is based on graphs, and learns the relationships among nodes and edges. In [10], Graph Convolutional Network was trained through supervised learning to solve a 7-bus problem, where buses were modeled as nodes, and transmission lines as edges. A difficulty is that training is not guaranteed to converge to provide good predictions, especially for large problems. Another method is reinforcement learning. In [11], a Q-learning-based agent is trained through offline and online supervised learning to solve a UC problem with 10 units. For large problems with many states and decisions, reinforcement learning may suffer from computation and performance difficulties. Reference [12] clustered historical demand values of the IEEE 118-bus system into several patterns, and trained Gated Recurrent Units (GRUs), each for a single pattern. GRUs were then used to predict solutions for new instances with different patterns of demand. This approach requires a large amount of historical data for high accuracy, leading to difficulties for large systems. In [13], regression trees and random forests were used to predict discrepancies between MILP solutions and LP-relaxed UC solutions. LP-relaxed solutions were then modified to obtain MILP solutions. This method was tested on MISO instances and had 78% error reductions as compared to that of directly using LP-relaxed solutions. It, nevertheless, may have difficulties when discrepancies between MILP solutions and LP-relaxed solutions are significant. Multiple ML methods, e.g., KNN, random forests, and support vector machine (SVM), were considered in [14] to predict unit on/off statuses for a 39-bus system. Dispatch problems were then solved to obtain generation levels. For large problems, it would be difficult to maintain the accuracy of the predicted commitment statuses.

III. Problem Formulation And Surrogate Lagrangian Relaxation

Section III-A summarizes the formulation of the deterministic hourly UC problem based on [18]. Section III-B presents the steps of SLR following [15].

A. UC Problem Formulation

Our formulation is based on [18]. For compactness of presentation, only equations that will be explicitly needed for later derivations are presented. Consider a power system consisting of *I* units each with *B* bid blocks, *L* transmission lines, and *N* nodes among *J* areas over *T* hours. The objective function to be minimized is the total operating cost of all the units over the *T* hours:

$$\sum_{i=1}^{I}\sum_{t=1}^{T}\left[C_{i,t}^{Start}u_{i,t}+C_{i,t}^{NL}x_{i,t}+\sum_{b=1}^{B}C_{i,b,t}^{E}p_{i,b,t}\right], \tag{1}$$

where the cost of unit i $(1 \le i \le I)$ at time t $(1 \le t \le T)$ consists of start-up cost $C_{i,t}^{Start}$, no-load cost $C_{i,t}^{NL}$, and generation cost $C_{i,b,t}^{E}$ per MW generation (assuming monotonically non-decreasing over blocks). The decision variables include binary start-up status $u_{i,t}$, binary commitment status $x_{i,t}$, and continuous generation level $p_{i,b,t}$ of bid block b $(1 \le b \le B)$.

Unit-level constraints include generation limits of each unit, generation limits of each bid block, start-up, ramp-up/down, and minimum up/down-time constraints. Generation limits of each bid block and minimum up/down time times are given in (2) and (5) of [18], respectively. For easy referencing later, the generation limits of unit *i* are provided below as:

$$P_i^{\min}x_{i,t} \le p_{i,t} \le P_i^{\max}x_{i,t}, \forall i, \forall t, \tag{2}$$

where $P_i^{\min}$ and $P_i^{\max}$ are the minimum and maximum generation limits, respectively; $p_{i,t}$ is the generation level of unit *i* at time

t, and it equals the sum of $\{p_{i,b,t}\}$ over blocks, i.e.,

$$\sum_{b=1}^{B} p_{i,b,t} = p_{i,t}, \forall i, \forall t, \tag{3}$$

Start-up constraints of unit i are given as:

$$u_{i,t} \geq x_{i,t} - x_{i,t-1}, \forall i, \forall t. \tag{4}$$

Ramp-up constraints of unit i are given as:

$$p_{i,t} - p_{i,t-1} \leq \left(\frac{R_i}{2} - P_{i,t}^{\min}\right) x_{i,t-1} + \left(\frac{R_i}{2} + P_{i,t}^{\min}\right) x_{i,t}, \forall i, \forall t, \tag{5}$$

where R_i is the ramp-up rate of unit i. Ramp-down is similarly formulated ((3) of [18]). The initial statuses of a unit (including the initial commitment status, the number of hours that the unit has been on or off, and the initial generation level) are given, and their values may vary across days.

System-wide constraints include system demand and transmission capacity constraints. For simplicity, system reserve constraints are not considered.

System demand constraints are given in (7) of [18], and are formulated as:

$$\sum_{i=1}^{I} p_{i,t} = \sum_{n=1}^{N} D_{n,t}, \forall t, \tag{6}$$

where $D_{n,t}$ is the demand at node n $(1 \leq n \leq N)$at time t. Transmission capacity constraints are given in (9) of [18], and are formulated as:

$$\underline{F}_l \leq f_{t,l} \leq \bar{F}_l, \text{with}$$
$$f_{t,l} = \sum_{n=1}^{N} \alpha_{n,l} \left(\sum_{i \in I_n} p_{i,t} - D_{n,t}\right), \forall t, \forall l, \tag{7}$$

where $f_{t,l}$ is the DC power flow on line l $(1 \leq l \leq L)$ at time t; $\underline{F}_l$ and $\bar{F}_l$ are the limits of line l; I_n denotes the set of units at node n; and $\alpha_{n,l}$ is the generation shift factor that indicates the change of $f_{t,l}$ w.r.t. a change in power injection at node n. Because a large number of transmission capacity constraints may be inactive, transmission lines can be filtered out in advance following (20) of [18] to reduce the computational requirements.

B. SLR Framework

This subsection briefly summarizes the SLR steps based on [15]. Only key equations are provided for easy derivations later.

1) Relaxing System-wide Constraints: System demand constraints (6) are relaxed by using multipliers λ_t, and transmission capacity constraints (7) by multipliers $\mu_{t,l}^{+}$ and $\mu_{t,l}^{-}$. The relaxed problem is:

$$\min_{u,x,p} L, \text{with}$$
$$L \equiv \left\{ \begin{array}{l} \sum_{i=1}^{I}\sum_{t=1}^{T}\left[C_{i,t}^{Start}u_{i,t} + C_{i,t}^{NL}x_{i,t} + \sum_{b=1}^{B} C_{i,b,t}^{E} p_{i,b,t}\right] \\ + \sum_{t=1}^{T}\left[\lambda_t\left(\sum_{n=1}^{N} D_{n,t} - \sum_{i=1}^{I} p_{i,t}\right) + \sum_{l=1}^{L}\mu_{t,l}^{+}\left(f_{t,l} - \bar{F}_l\right)\right. \\ \left. + \sum_{l=1}^{L}\mu_{t,l}^{-}\left(\underline{F}_l - f_{t,l}\right)\right] \end{array} \right\}, \tag{8}$$

subject to generation limits (2)–(3), start-up constraints (4), ramp-up constraints (5), and all other unit-level constraints.

2) Formulating Subproblems: Units are divided into J subproblems based on their areas. The set of units belonging to subproblem j is denoted as I_j. By collecting all the terms in L belonging to I_j , subproblem j at iteration k is:

$$\min_{u,x,p} L_j^k, \text{with}$$
$$L_j^k \equiv \left\{ \begin{array}{l} \sum_{i \in I_j}\sum_{t=1}^{T}\left[C_{i,t}^{Start}u_{i,t} + C_{i,t}^{NL}x_{i,t} + \sum_{b=1}^{B} C_{i,b,t}^{E} p_{i,b,t}\right] \\ + \sum_{t=1}^{T}\left(-\sum_{i \in I_j}\lambda_t^k p_{i,t} + \sum_{l=1}^{L}\left(\mu_{t,l}^{k,+} - \mu_{t,l}^{k,-}\right)\right. \\ \left.\left(\sum_{n=1}^{N}\sum_{i \in (I_n \cap I_j)}\alpha_{n,l}p_{i,t}\right)\right) \end{array} \right\}, \tag{9}$$

subject to all unit-level constraints.

For a given set of multipliers and initial statuses of units, subproblems are solved to obtain good-enough solutions that are feasible with respect to unit-level constraints and satisfy the surrogate optimality condition (SOC) (12) of [15]):

$$\tilde{L}\left(u_j^k, x_j^k, p_j^k, \lambda^k, \mu^k\right) < \tilde{L}\left(u_j^{k-1}, x_j^{k-1}, p_j^{k-1}, \lambda^k, \mu^k\right), \tag{10}$$

where $\tilde{L}(u_j^k, x_j^k, p_j^k, \lambda^k, \mu^k)$ is the "surrogate dual value", which is L of (8) evaluated with decision variables of area j at iteration k, and with decision variables of other areas at iteration $k-1$. Similarly,$\tilde{L}(u_j^{k-1}, x_j^{k-1}, p_j^{k-1}, \lambda^k, \mu^k)$ is L evaluated with all decision variables at iteration k-1. Since SLR subproblems are independent of each other for a given set of multipliers, the above SOC is satisfied if $L_j^k < L_j^{k-1}$ when solving subproblem j at iteration k. In our previous results [17], [18], [19], good-enough subproblem solutions to (9) were obtained by using OO or B&C. When using OO, solutions from previous iterations were used as "solution candidates" to be modified by heuristics such as neighborhood search to obtain good-enough solutions.

3) Coordinating Subproblems: If a good-enough solution is obtained after solving a subproblem, multipliers are updated following (14)–(15) of [15]. If no good-enough solution is obtained, SLR skips the updating process, and moves on to solve the next subproblem. The above process leads to the convergence of multipliers λ as proved in Theorem 2.1 of [15]. The iterative process of subproblem-solving and multiplier updating terminates when the multipliers converge, or after each subproblem has been solved a certain number of times, or when a time limit is reached. Since system-wide constraints are relaxed, directly putting subproblem solutions together generally does not form a solution feasible to the original UC. Heuristics are thus used to obtain solutions feasible to the original formulation. The heuristic used in Section VI is to fix all binary variables at subproblem solution values and then solve the resulting LP-relaxed problem for generation levels.

IV. Offline Subproblem Supervised Learning

Within SLR, subproblems are solved for a given set of multipliers. If subproblems are solved by using B&C or OO, there is no learning. Our idea is to use DNN to learn subproblem solutions. DNN architecture and dimensionality reduction are

presented in Section IV-A. For effective training, the distribution of random multipliers is innovatively specified based on the locations of "jumps" of binary decisions in Section IV-B. In IV-C, a loss function including a squared error part and a regularization part is designed for offline supervised learning. In IV-D, predictions are adjusted for feasibility, and graceful degradation is accomplished for unfamiliar cases by using OO or B&C as a backup.

A. DNN Architecture and Dimensionality Reduction

In this subsection, ML is used for the first time to learn subproblem solutions. For simplicity, the set of units, unit characteristics (including start-up, no-load, and generation costs, generation limits, ramp rates, and minimum up/down times), and capacities of transmission lines are assumed constant across days. Only system demand and unit initial statuses (including the initial commitment status $x^0_{i,t}$, the number of hours that the unit has been on T^{On}_i or off T^{Off}_i, and the initial generation level $p^0_{i,t}$) change from day to day. Under these simplifying assumptions, ML does not require a complicated architecture. A simple fully-connected multilayer perceptron with three hidden layers is thus adopted rather than advanced DNN architectures. The task of the DNN is to predict good-enough subproblem solutions based on a given set of multipliers and unit initial statuses.

In view of the large numbers of multipliers and variables, using all of them, however, would require a large DNN that may be hard to train. Take the IEEE 118-bus system with 54 units and 186 transmission lines over 24 hours to be considered in Section VI as an example. Although the number of demand multipliers λ_t is only 24, the number of transmission capacity multipliers ($\mu^+_{t,l}$ and $\mu^-_{t,l}$) is 8928 ($2 \times 24 \times 186$). It would be even larger for the Polish 2383-bus system. These multipliers are too many for effective training. By examining (9), for unit i at node n, i.e., $i \in I_n$, these multipliers can be grouped as "aggregated" multipliers:

$$\lambda^A_{i,t} \equiv \lambda_t - \sum_{l=1}^{L} \alpha_{n,l} \left(\mu^+_{t,l} - \mu^-_{t,l}\right), \forall n, \forall i \in I_n \cap I_j, \forall t. \quad (11)$$

These aggregated multipliers are individualized w.r.t. units (or actually nodes). Although aggregated multipliers have different impacts on individual units, from a system perspective, they have the same range for a given t. Consequently, the required size of the DNN is much reduced. For the above IEEE 118-bus system, the number of aggregated multipliers for a unit is 24, and the total number of aggregated multipliers for all the units is 1296 (54×24).

For the output, since $\{p_{i,t}\}$ and $\{u_{i,t}\}$ can be easily derived from $\{x_{i,t}\}$ and $\{p_{i,b,t}\}$ using (3) and (4), there is no need to predict them separately. Therefore, for dimensionality reduction, the output only contains $\{x_{i,t}\}$ (binary) and $\{p_{i,b,t}\}$ (continuous). The DNN is summarized in Fig. 1. With multipliers as shadow prices, predictions can be interpreted as the power that suppliers would be willing to provide based on the market demand and supply relationship.

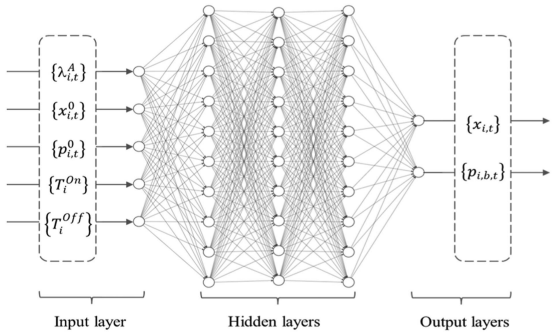

Fig. 1. Deep neural network.

B. Specification of Multiplier Distributions

When preparing training data, a simple-minded way is to solve UC instances with various system demand and unit initial statuses by using SLR+B&C, and collect $\{\lambda^A_{i,t}\}$ and subproblem solutions. This, however, may not be able to capture the impacts of various combinations of aggregated multipliers on solutions because unit on/off statuses and generation levels in subproblems are essentially affected by Lagrangian multipliers rather than the demand itself.. Subproblems are thus directly solved with various combinations of aggregated multipliers and unit initial statuses. This, nevertheless, is challenging in view that binary decisions $\{x_{i,t}\}$ jump from "off (0)" to "on (1)" at a few discrete points (i.e., breakpoints) of $\{\lambda^A_{i,t}\}$, as opposed to varying continuously across $\{\lambda^A_{i,t}\}$. The specification of aggregated multiplier distributions to be sampled from must therefore be carefully designed for effective training. The focus should be around breakpoints, which, however, are difficult to specify. In the following, a scheme is developed so that probabilities of aggregated multipliers sampled from for training will concentrate around approximated breakpoints.

The range of $\{\lambda^A_{i,t}\}$ is first determined based on testing experience, e.g., $[-30, 100]$ for Examples 1 and 2 in Section VI. A single unit problem without time-coupling constraints such as ramp rates and minimum up/down times is first considered. For a particular hour t, if the unit is off at t-1, $x_{i,t}$ jumps from 0 to 1 at a breakpoint of $\lambda^A_{i,t}$. The exact value of this breakpoint is difficult to obtain since it depends on the values of the aggregated multipliers for the next few hours so as to amortize the start-up and no-load costs. Nevertheless, an approximated breakpoint can be quickly obtained as the full-load average cost per MW with the start-up and no-load costs averaged over P^{Max}_i. With the presence of time-coupling constraints, the values of breakpoints generally do not change significantly from those without time-coupling constraints except during minimum up/down times where $x_{i,t}$ cannot be changed. As a result, the values of breakpoints without time-coupling constraints can still be used.

Since units are divided to form J subproblems based on their areas as presented in Section III-B, there are generally multiple units within a subproblem. In view that units are independent after the relaxation of system-wide constraints, their breakpoints can be simply grouped together for a subproblem. As a result, the number of breakpoints for a subproblem equals the number of units in the subproblem. When multiple breakpoints are close to each other, e.g., having the same integer part, the average

value of these points is used instead. For each subproblem, an overlapping truncated normal distribution is designed for every t. Each component of the truncated normal distribution has a breakpoint as its mean, and its variance is empirically selected so that the probability from the smallest to the largest breakpoints for the overall distribution is larger than 0.95. Offline training cases are then created based on aggregated multipliers sampled from these distributions and with unit initial commitment statuses randomized to be on or off. If a unit is initially on, the number of hours it has been on is randomly sampled from a uniform distribution, and its initial generation level is set to be $P_i^{\min}$ for simplicity. Otherwise, its initial generation level is zero, and the number of hours it has been off is randomly sampled from a uniform distribution. For these offline training cases, B&C is used to obtain subproblem solutions that are within a certain gap (e.g., default limit of Gurobi [22]) as the learning targets.

C. Specially Designed Loss Function and Offline Learning

Based on the training data thus created, the DNN can be trained offline by using the learning targets. There are several difficulties. First, in view of the existence of many types of unit-level constraints, predicting good-enough subproblem solutions (i.e., feasible w.r.t. unit-level constraints and satisfying (10)) is still challenging. Also, it is difficult to predict binary $\{x_{i,t}\}$ when DNN variables are all real valued. Finally, values of $\{p_{i,b,t}\}$ and $\{x_{i,t}\}$ are not in the same order of magnitude. Using the same weight in a loss function may lead to a biased learning – focusing on $\{p_{i,b,t}\}$ while ignoring $\{x_{i,t}\}$. To address the above, a loss function considering both target values and constraint satisfaction is specially designed. It includes two parts: an error part as a weighted sum of squared errors of all variables, and a regularization part considering the violations of unit-level constraints as:

$$\begin{aligned} loss^{off} \equiv & \sum_{i=1}^{I}\sum_{t=1}^{T}\left\{\left(\tilde{x}_{i,t}-x_{i,t}^{*}\right)^2+\frac{1}{B}\sum_{b=1}^{B}\left(\frac{(\tilde{p}_{i,b,t}-p_{i,b,t}^{*})}{P_i^{\text{Max}}}\right)^2\right\} \\ & +\frac{1}{I\cdot T}\sum_{i=1}^{I}\sum_{t=1}^{T}\left\{\left[\tilde{x}_{i,t}\left(1-\tilde{x}_{i,t}\right)\right]^2\right. \\ & +\left[\max\left(\left(\sum_{b=1}^{B}\frac{\tilde{p}_{i,b,t}}{P_i^{\text{Max}}}-\tilde{x}_{i,t}\right),0\right)\right]^2 \\ & +\left[\max\left(\left(\frac{P_i^{\text{Min}}\tilde{x}_{i,t}}{P_i^{\text{Max}}}-\sum_{b=1}^{B}\frac{\tilde{p}_{i,b,t}}{P_i^{\text{Max}}}\right),0\right)\right]^2 \\ & +\left[\max\left(\left(\frac{\tilde{p}_{i,t}-\tilde{p}_{i,t-1}-R_i\tilde{x}_{i,t-1}}{P_i^{\text{Max}}}\right)\right.\right. \\ & \left.\left.-\frac{(2P_i^{\text{Min}}+R_i)(\tilde{x}_{i,t}-\tilde{x}_{i,t-1})}{2P_i^{\text{Max}}}\right),0\right)\Bigg]^2 \\ & \left.+\text{violations of ramp-down comstraints and min-up/down constraints}\right\}. \end{aligned} \tag{12}$$

In (12), $\tilde{x}_{i,t}$ and $\tilde{p}_{i,b,t}$ are the output of DNN, and $x_{i,t}^{*}$ and $p_{i,b,t}^{*}$ are their target values obtained from B&C in advance. The first and second terms are the weighted sum of squared errors to learn the target values of all variables. To avoid biased learning focusing on $\{p_{i,b,t}\}$, the second term is normalized by $P_i^{\max}$ to be in the same order of magnitude as $\{x_{i,t}\}$, and a smaller weight based on the number of bid blocks is used. The third term is a regularization term for binary $x_{i,t}$. Since ML predicts a continuous value of $\tilde{x}_{i,t}$ in-between 0 and 1, this term penalizes

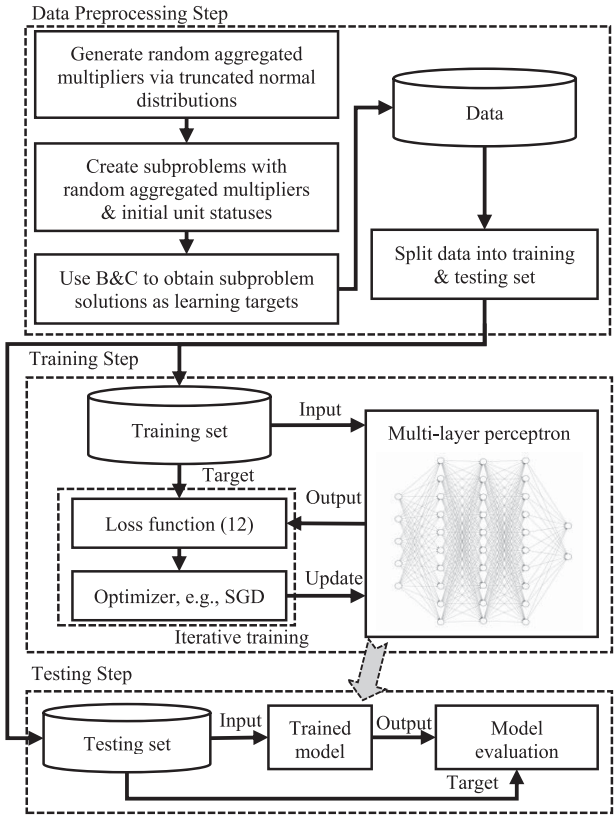

Fig. 2. Flowchart of offline supervised training.

$\tilde{x}_{i,t}$ when it is not 0 or 1. The fourth and fifth terms penalize the violations of generation limits (2). The sixth term penalizes the violations of ramp-up constraints (5). The penalties for the violations of ramp-down and minimum up/down-times are similarly designed, but not explicitly presented here for compactness. The gradient of the loss function (12) w.r.t. DNN weights can then be derived to update these weights via back-propagation.

Offline learning is implemented via Pytorch [23], and the steps, including data preprocessing, training, and testing are described in Fig. 2. Following Pytorch's default setting, DNN weights are initialized at 0.5, and no bias is used. The learning rate is selected based on a small experiment where the DNN performance is tested across different learning rates from 0.000001 to 10 at logarithmic intervals, as will be presented in Section VI.

D. Feasibility Enhancement and Graceful Degradation for Unfamiliar Cases

As mentioned in III.B, subproblem solutions should be good enough, i.e., feasible w.r.t. unit-level constraints and satisfying the SOC (10). Regarding feasibility, although infeasibility is penalized in (12), DNN predictions may still be infeasible. If so, they are adjusted by using heuristics as follows. For unit i, $\tilde{x}_{i,t}$ is first rounded if it is fractional. Values of $\{\tilde{p}_{i,b,t}\}$ are forced to be within their limits. Also, if the rounded $\tilde{x}_{i,t}$ is 0, $\{\tilde{p}_{i,b,t}\}$ for all the blocks are set to be 0. Variables $\{u_{i,t}\}$ and $\{p_{i,t}\}$ can be derived based on (3) and (4). Time-coupling constraints are then checked, and $\{p_{i,t}\}$ are modified by using heuristics as needed. For example, if the change of generation levels from t to $t+1$

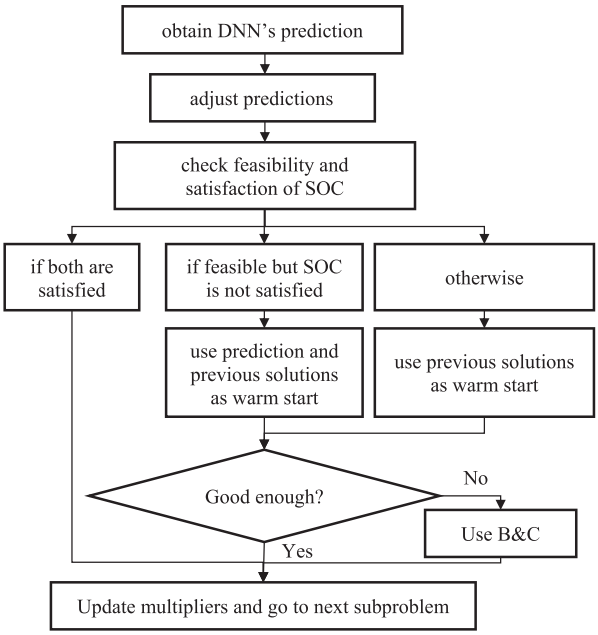

Fig. 3. Flowchart of using ML to solve a subproblem.

exceeds the ramp-up rate, then the generation at $t+1$ is reduced to satisfy the ramp-up constraint.

Even with the above enhancement, predictions might still be infeasible. Also, feasible predictions might not satisfy the SOC. For both situations, graceful degradation is accomplished by using OO or B&C as a backup, with a slight increase in computational time. OO is first called upon to search for good-enough solutions by modifying feasible predictions or solutions from the previous iterations using heuristics, e.g., the one embedded in Gurobi. If OO still cannot obtain good-enough solutions, B&C is called upon as the last resort to solve the subproblem. After a good-enough subproblem solution is obtained, the multipliers are updated, and the next subproblem is solved. If no good-enough solution is obtained, SLR moves to solve the next subproblem as presented in III.B. Although SLR requires iterations to converge, the total computational burden is much reduced by introducing DNN to learn and to predict subproblem solutions. The above procedure is summarized in Fig. 3.

V. Online Self-Learning With Positive/Negative Cases

To effectively utilize subproblem solutions available from daily operations, supplementary online self-learning is considered. Section V-A presents learning with positive cases. Section V-B presents learning with negative cases.

A. Online Self-Learning With Positive Cases

It is difficult for offline learning to cover all possible cases, although multiplier distributions are carefully designed. To improve performance, our idea is to utilize subproblem solutions available from daily operations for supplementary learning. When solving a new UC instance, each subproblem is iteratively solved multiple times following Section IV-D. Among those iterations, there are "positive cases" where DNNs successfully predict good-enough subproblem solutions. For such cases, once a good enough prediction is obtained, it can be used as a learning target for online learning, and the loss function for offline learning described in Section IV-C is used to derive the gradient to update DNN weights.

B. Online Self-Learning With Negative Cases

There are also "negative cases" where a DNN fails to provide good-enough subproblem solutions. Without good-enough solutions, can the DNN still learn? Consider first the situation where a DNN prediction is feasible but failed to satisfy the SOC. The goal of satisfying the SOC (10) is innovatively converted to the following loss function for unsupervised learning:

$$loss^{-} \equiv \frac{\tilde{L}(u_j^k, x_j^k, p_j^k, \lambda^k, \mu^k) - \tilde{L}(u_j^{k-1}, x_j^{k-1}, p_j^{k-1}, \lambda^k, \mu^k)}{\left|\tilde{L}(u_j^k, x_j^k, p_j^k, \lambda^k, \mu^k) - \tilde{L}(u_j^{k-1}, x_j^{k-1}, p_j^{k-1}, \lambda^k, \mu^k)\right|}. \tag{13}$$

when a failed DNN prediction is feasible, it can be used to calculate the surrogate dual value $\tilde{L}(u_j^k, x_j^k, p_j^k, \lambda^k, \mu^k)$, and the subgradient of (13) w.r.t DNN weights can be derived. These weights can thus be updated online via back-propagation to reduce the left-hand side of the SOC for better predictions in the future.

For the second situation where a DNN prediction is not feasible, the prediction is ignored and not used for online self-learning. For both situations (a feasible prediction but failing to satisfy the SOC, and an infeasible prediction), OO or B&C are called upon as described in Section IV-D, and good-enough subproblem solutions can still be obtained. These solutions can also be used for supplementary online learning similar to that for positive cases.

With online self-learning from both positive and negative cases in daily operations, DNNs are expected to perform better and better. Nevertheless, since online learning is executed once or a few times at each subproblem-solving iteration when facing a new instance, the impacts might not be that significant as compared to offline learning. Good-enough solutions from self-learning can be collected and used in a new round of offline supervised learning to further improve the DNNs. The transition between offline supervised learning and online self-learning is achieved by a simple switching of the loss functions (12) and (13).

By embedding offline learning (Section IV) and online learning (Section V) within SLR, the new method can learn from the past. It also inherits many good properties of SLR, e.g., guaranteeing convergence of multipliers, generating near-optimal solutions with quantifiable quality, interpretability of solutions based on the shadow price concept of multipliers, and allowing fast warm-start from previous runs. Consequently, our method holds much promise for solving complicated UC or other MILP problems.

VI. Numerical Testing

Our method was implemented in Python 2.8, Gurobi 9.1.2, and Pytorch 1.9.1+cu111 on a PC with Intel Xeon Gold 6248R CPU @3.0 GHz, 190GB RAM, and NVIDIA Quadro RTX 6000.

TABLE I
CHARACTERISTICS OF 4 UNITS

Unit	P^{min}	P^{max}	$C^{Start}_{i,t}$	$C^{NL}_{i,t}$	$C^{E}_{i,b,t}$	RR	Min up	Min down
1	0	30	40	32	27	10	2	2
2	30	80	100	7	14.5	12.5	1	1
3	10	25	50	10.5	15.5	7.5	2	2
4	15	40	30	18	18	20	0	0

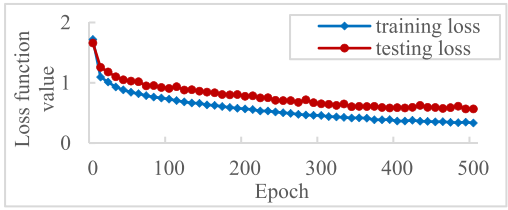

Fig. 4. loss function values of versions that select multipliers from truncated normal dist.

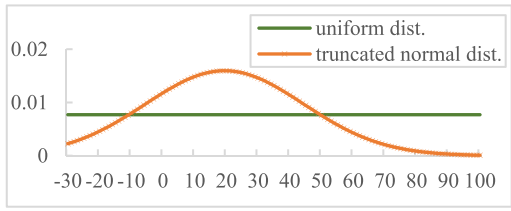

Fig. 5. Probability densities of truncated normal dist. and uniform dist.

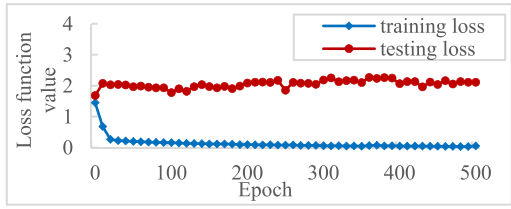

Fig. 6. loss function values of the version with uniform dist.

Three examples are considered in this section. In Example 1, a small 4-unit UC problem is used to demonstrate the ability of ML to provide good-enough subproblem solutions. In Example 2, the IEEE 118-bus system is considered to show that ML can speed up SLR's process while maintaining near-optimality of the overall solutions. With continual learning, ML provides an increasing percentage of good-enough subproblem solutions, leading to further reductions in subproblem-solving time. In Example 3, the Polish 2383-bus system is used to demonstrate the ability of our approach to solving large problems.

A. Example 1

Consider a small 4-unit problem over 5 hours. For simplicity, each unit has 1 bid block, and no transmission line is considered. In the absence of transmission capacity multipliers, aggregated multipliers are just demand multipliers $\{\lambda_t\}$. Table I provides the characteristics of the units. The problem is decomposed into four subproblems, one for each unit. Only offline supervised learning is examined.

Offline learning: For each subproblem, 12 k training cases were generated based on a distribution of aggregated multipliers over [−30, 100] as presented in Section IV-B. To measure the performance of DNNs, 75 new UC instances (3 k testing cases) were generated based on given historical demand data, with each period's demand varying with a random factor within the range of [0.8, 1.2] and unit initial statuses randomized as described in Section IV-B. For offline supervised learning, DNN weights were initialized at 0.5, and the learning rate was selected as 0.0003 based on the design described in Section IV-C.

For all the four subproblems, both training and testing losses were reduced across epochs, and the total training time was about 16 minutes. The losses for subproblem 1 across 500 epochs are shown in Fig. 4 as an example. To demonstrate the effectiveness of the multiplier distribution design, a DNN with 12 k training cases based on a uniform distribution over [−30, 100] and the 3 k testing cases described above were examined for subproblem 1 as a comparison. The two probability density functions are plotted in Fig. 5. The training and testing losses based on the uniform distribution are shown in Fig. 6. It can be seen that the testing loss does not reduce across epochs, demonstrating the value of the distribution design of IV.B.

Solving the entire UC: Ten new UC instances with random system demand and unit initial statuses were considered. They were solved by using our method of SLR+ML. The stopping criterion was a relative gap of 0.1% between the feasible solution cost and the best-known lower bound (obtained by B&C in advance). For all units, more than 90% of DNN predictions were feasible, and about 40% of these feasible ones were good enough and were directly used to update multipliers. Although the rest of the feasible predictions were not good enough, they were used as "solution candidates" in OO to quickly search for good-enough solutions as presented in Section IV-D. After using OO, more than 90% of iterations had good enough solutions, and B&C was called upon for the rest of the iterations. SLR with subproblems solved by using OO (SLR+OO, [17]) was also used to solve these instances for comparison purposes. Our method obtained solutions with an average relative gap of less than 0.001%, the same as what SLR+OO obtained. The results thus demonstrate that ML with OO or B&C as a backup provides good-enough subproblem solutions, and that our method can obtain overall near-optimal solutions. Since these problems were small, the CPU times for both methods were less than 1 min for each instance.

B. Example 2

The IEEE 118-bus system [24] is considered over 24 hours. There are 54 units each with 6 bid blocks, and 186 transmission lines. The problem is decomposed into six subproblems containing 6, 11, 8, 10, 10, and 9 units each.

Offline learning: Consider first the situation with offline supervised learning only (SLR+ML (1)). Following the procedure of Section IV-B, there were 5, 7, 6, 7, 7, and 6 approximated

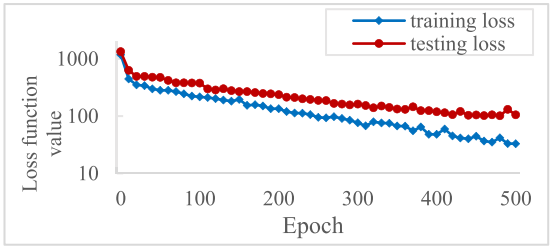

Fig. 7. loss function values in offline training.

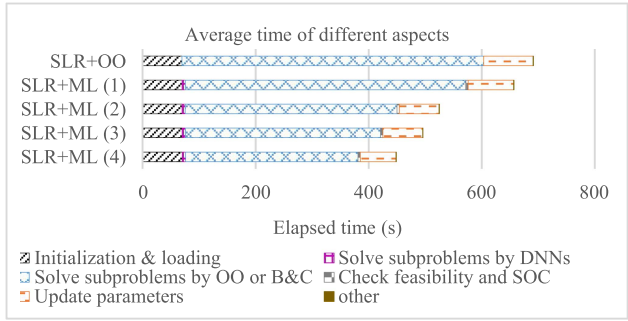

Fig. 8. Average elapsed time spent on various functions SLR+ML and SLR+OO for 118-bus system.

breakpoints found for six subproblems, and six aggregated multiplier distributions were created over [−30, 100], one for each subproblem. For each subproblem, 1 k offline training cases and 200 offline testing cases were then generated. The DNN weights were initialized at 0.5, and the learning rate was 0.0003 based on the design presented in Section IV-C. The training time was around 20 mins, and both training and testing losses were reduced across epochs during the offline training. Although the value of the testing loss was not as low as that of the training loss, it was of the same magnitude. The losses for subproblem 1 across 500 epochs are shown in Fig. 7 as an example.

Solving the entire UC: To measure the performance of our method on solving new instances in daily operations, 100 new UC instances with random system demand and unit initial statuses as described in Example 1 were considered. For these instances, our method obtained near-optimal solutions with an average gap of 0.35% after around 660 seconds. In our method, ML predicted the subproblem solutions within milliseconds per minor iteration, and around 8% of these predictions were good enough. For the iterations in which ML did not provide good-enough predictions, OO and B&C were used as backups, which maintained the quality of the overall solution. The result shows our method has the ability to tackle the challenges of unfamiliar UC instances in daily operations.

Further improvement through online learning: To further improve performance, online self-learning was applied. Forty-five new instances with random system demand and unit initial statuses were considered for self-learning in this second situation (SLR+ML (2)). Each subproblem was solved for 40 iterations, leading to 1.8 k (=40 × 45) self-learning cases. As presented in Section V, the transition from offline supervised learning to online self-learning was achieved at a switching of the loss functions. Although DNN weights experienced small fluctuations at the transition, they generally converged. The total training time, including both offline and online learning, was around 50 mins. Our method was then used to solve the 100 same new UC instances, and obtained solutions with an average gap of 0.38%. The average percentage of good-enough predictions by ML was increased from 8% to 24%, leading to a reduction of average CPU time from 660 to 530 seconds. It shows that online training can further improve the performance.

Two more situations were considered to demonstrate the impact of continual online self-learning. The third situation (SLR+ML (3)) had 2 k offline cases plus 45 online instances (i.e., 1.8 k cases) for a total training time of 1 hour; and the fourth situation (SLR+ML (4)) had 2 k offline cases plus 150 online instances (i.e., 6 k cases) for a total training time of 2 hours. When solving the same 100 new UC instances, the third situation obtained solutions with an average gap of 0.33% in 495 seconds, and the percentages of good-enough predictions by ML were increased to 26%. The fourth situation obtained solutions with an average gap of 0.37% in 450 seconds, and ML provided 32% good-enough predictions, but only spent around 1.5% of the total time. For comparison purposes, SLR+OO was also used to solve these 100 new instances and obtained solutions with an average gap of 0.33% in 690 seconds. The average times spent on various functions by our method for the four situations and by SLR+OO are analyzed in Fig. 8. From the figure, it can be seen that by integrating ML in SLR, the subproblem-solving process is accelerated as compared to that of SLR+OO. This is because when ML provides good-enough predictions, OO or B&C are not needed. Moreover, with continual learning, ML provides a higher percentage of good-enough solutions, and requires less calling on OO or B&C, leading to further reductions in subproblem-solving time.

C. Example 3

The Polish 2383-bus system [25] is considered over 24 hours. There are 327 units each with 6 bid blocks, and 2895 transmission lines. To reduce the complexity, inactive transmission lines are filtered out in advance as presented in Section III-A. After filtering, the number of transmission lines remaining active is 397. For simplicity, unit initial statuses are assumed given. The problem is decomposed into five subproblems containing 46, 43, 95, 96, and 47 units each.

Offline learning: Consider first the situation with offline supervised learning only (SLR+ML (1)). When preparing offline training data, it is difficult to use breakpoints of so many units to generate multiplier distributions. For simplicity, five distributions were generated over [−100, 500] based on 15 approximated breakpoints that were obtained by sorting units' full-load average costs and selecting values with equally spaced intervals. For each subproblem, 10 k offline cases were generated. The DNN weights were initialized at 0.5, and the learning rate was 0.00002 based on the design of Section IV-C. For all the five subproblems, both training and testing losses were reduced across epochs, and although the value of the testing loss was not

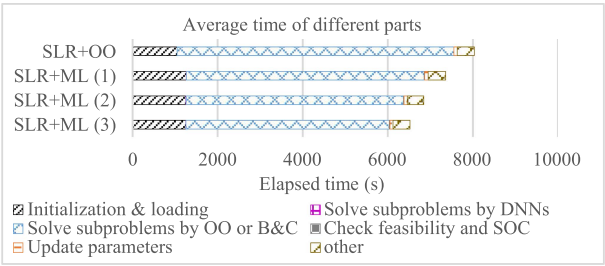

Fig. 9. Average elapsed time spent on various functions SLR+ML and SLR+OO for the Polish 2383-bus system.

as low as that of the training loss, it was of the same magnitude. The training time was around 2 hours.

Solving the entire UC: To measure performance, a new UC instance with random system demand as described in Example 1 was considered, but with a factor within the range of [0.9, 1.1] based on the consideration of feasibility. Our method obtained a near-optimal solution with a gap of 0.026% after 7,300 s, and ML provided 14% good enough subproblem solutions (predicted within milliseconds) over 50 iterations.

Further improvement through online learning: To further improve performance, two more situations with online self-learning were considered. The second situation (SLR+ML (2)) had 10 k offline cases plus 5 k online cases (i.e., 125 online instances, each with its every subproblem solved for 40 iterations) for a total training time of around 4 hours; and the third situation (SLR+ML (3)) had 10 k offline cases plus 10 k online cases (i.e., 250 online instances) for a total training time of 5 hours. When solving the new UC instance of the first situation, the second situation obtained a near-optimal solution with a gap of 0.011% after 6,800 s, and had 22% good enough subproblem solutions from ML. The third situation obtained a near-optimal solution with a gap of 0.010% after 6,500 s, and had 28% good enough subproblem solutions from ML. For comparison purposes, SLR+OO was also used to solve the new instance, and it obtained solutions with a near-optimal solution with a gap of 0.011% after 8,000 s. The time spent on various functions by our method and by SLR+OO are shown in Fig. 9. It is easy to see that with good enough subproblems provided by ML, the subproblem-solving time is smaller as compared to that of SLR+OO. Also, similar to that in Example 2, continual learning leads to an increasing percentage of good-enough solutions, and further reduction of subproblem-solving time. The above results demonstrate that our new method is robust and can be used to solve large UC problems. With further learning, the percentage of good-enough subproblem solutions from ML is expected to further increase.

For this first attempt to integrate ML and SLR, our focus is on demonstrating that ML can learn and predict good enough subproblem solutions, and that continual learning would lead to a higher percentage of good enough subproblem solutions, as opposed to showing that SLR+ML outperforms B&C in terms of solution quality or computation efficiency. Although B&C finds solutions with an average gap of 0.01% after 60 seconds for Example 2; and finds solutions with a gap of 0.008% after 1412 seconds for Example 3, there is no learning. When facing very complex UC problems, e.g., MISO's problems where B&C suffers from poor performance [17] and [19], our approach would present a promising new direction.

VII. Conclusion

This paper presents a synergistic integration of machine learning and Surrogate Lagrangian Relaxation for deterministic UC problems. Offline supervised learning and online self-learning are seamlessly unified to learn and predict good-enough subproblem solutions. Results demonstrate that ML learns to predict good enough subproblem solutions. With continual learning, the percentage of good-enough subproblem solutions from ML keeps increasing, leading to a faster subproblem-solving process while SLR maintains near-optimality of the overall solutions. Also, when facing unfamiliar cases, the quality of the overall solutions is maintained via the embedded OO or B&C as a backup. Our method thus opens a new direction for integrating ML and mathematical optimization in solving complicated MILP problems in power systems and beyond.

Acknowledgment

The author Peter B. Luh, who was the Academic supervisor of this project, tragically passed away in November 2022. By integrating mathematical optimization and machine learning in an innovative manner, Dr. Luh envisioned and actively modernized the way how difficult and important mathematical programming problems, such as the Unit Commitment considered in this paper, are solved. As a tribute to our dear friend and mentor, the remaining coauthors dedicate this paper to commemorating Dr. Luh's contributions and the legacy. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the MISO, NSF and NTU.

References

[1] J. E. Mitchell, "Branch-and-cut," in *Wiley Encyclopedia of Operations Research and Management Science*. Hoboken, NJ, USA: Wiley, 2010.
[2] E. B. Khalil, P. L. Bodic, L. Song, N. George, and D. Bistra, "Learning to branch in mixed integer programming," in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, pp. 724–731.
[3] A. Lodi and G. Zarpellon, "On learning and branching: A survey," *Data Sci. Real-Time Decis.-Mak., Top*, vol. 25, no. 2, pp. 207–236, Jun. 2017.
[4] A. S. Xavier, F. Qiu, and S. Ahmed, "Learning to solve large-scale security-constrained unit commitment problems," *Informs J. Comput.*, vol. 33, no. 2, pp. 739–756, Oct. 2020.
[5] A. Jiménez-Cordero, J. M. Morales, and S. Pineda, "Warm-starting constraint generation for mixed-integer optimization: A machine learning approach," *Knowl.-Based Syst.*, vol. 253, 2022, Art. no. 109570.
[6] Y. Yang and L. Wu, "Machine learning approaches to the unit commitment problem: Current trends, emerging challenges, and new strategies," *Electricity J.*, vol. 34, no. 1. Jan. 2021, Art. no. 106889.
[7] Y. Yang, X. Lu, and L. Wu, "Integrated data-driven framework for fast SCUC calculation," *IET Gener., Transmiss. Distrib.*, vol. 14, no. 24, pp. 5728–5738, Dec. 2020.
[8] Y. Zhou, Q. Zhai, L. Wu, and M. Shahidehpour, "A data-driven variable reduction approach for transmission-constrained unit commitment of large-scale systems," *J. Modern Power Syst. Clean Energy*, vol. 11, no. 1, pp. 254–266, Jan. 2023.
[9] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.

[10] P. S. Gaikwad, "Using graph convolutional network and message passing neural networks for solving unit commitment and economic dispatch in a day ahead energy trading market based on ERCOT nodal model," M.S. thesis, Univ. Texas at Arlington, Arlington, TX, USA, 2020.
[11] P. G. Latha, "An efficient machine learning algorithm for unit commitment problem," *Int. J. Appl. Eng. Res.*, vol. 13, no. 3, pp. 135–141, Nov. 2018.
[12] N. Yang et al., "Deep learning-based SCUC decision-making: An intelligent data-driven approach with self-learning capabilities," *IET Gener., Transmiss. Distrib.*, vol. 16, no. 4, pp. 629–640, 2022.
[13] X. Lin, Z. J. Hou, H. Ren, and F. Pan, "Approximate mixed-integer programming solution with machine learning technique and linear programming relaxation," in *Proc. 3rd Int. Conf. Smart Grid Smart Cities*, 2019, pp. 101–107.
[14] T. Iqbal, H. U. Banna, A. Feliachi, and M. Choudhry, "Solving security constrained unit commitment problem using inductive learning," in *Proc. IEEE Kansas Power Energy Conf.*, 2022, pp. 1–4.
[15] M. A. Bragin, P. B. Luh, J. H. Yan, N. Yu, and G. A. Stern, "Convergence of the surrogate Lagrangian relaxation method," *J. Optim. Theory Appl.*, vol. 164, no. 1, pp. 173–201, Apr. 2015.
[16] X. Guan, P. B. Luh, H. Yan, and J. A. Amalfi, "An optimization-based method for unit commitment," *Int. J. Elec. Power Energy Syst.*, vol. 14, no. 1, pp. 9–17, Feb. 1992.
[17] J. Wu, P. Luh, Y. Chen, M. Bragin, and B. Yan, "A novel optimization approach for sub-hourly unit commitment with large numbers of units and virtual transactions," *IEEE Trans. Power Syst.*, vol. 37, no. 5, pp. 3716–3725, Sep. 2022.
[18] B. Yan et al., "Effects of tightening unit-level and system-level constraints in unit commitment," in *Proc. IEEE Power Energy Soc. Gen. Meet.*, 2019, pp. 1–5.
[19] X. Sun, P. B. Luh, M. A. Bragin, Y. Chen, J. Wan, and F. Wang, "A novel decomposition and coordination approach for large day-ahead unit commitment with combined cycle units," *IEEE Trans. Power Syst.*, vol. 33, no. 5, pp. 5297–5308, Sep. 2018.
[20] M. A. Bragin, P. B. Luh, B. Yan, and X. Sun, "A scalable solution methodology for mixed-integer linear programming problems arising in automation," *IEEE Trans. Automat. Sci. Eng.*, vol. 16, no. 2, pp. 531–541, Apr. 2019.
[21] Y. C. Ho, Q. Zhao, and Q. Jia, *Ordinal Optimization: Soft Optimization For Hard Problems*. NewYork, NY, USA: Springer, 2007.
[22] Gurobi Optimization Inc, "Gurobi optimizer reference manual," 2014. [Online]. Available: http://www.gurobi.com/
[23] "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32. New York, NY, USA: Curran Associates, pp. 8024–8035, [Online]. Available: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf
[24] "Electric power system analysis & nature-inspired optimization algorithms, test system," Mar. 2021. [Online]. Available: https://al-roomi.org/component/tags/tag/test-system
[25] CASE2383WP, the power flow data for Polish system, [Online]. Available: https://matpower.org/docs/ref/matpower5.0/case2383wp.html